



ಜಿ. ಎಂ. ವಿಶ್ವವಿದ್ಯಾಲಯ

GM UNIVERSITY

P. B. Road, Davanagere – 577 006 KARNATAKA | INDIA

Faculty of Computing and IT Master in Computer Applications

Course: Ethical Hacking

Semester: IISem

Academic Year: 2024-25

Session 01-Assignment

ASSIGNMENT Title: SQL INJECTION

STUDENT DETAILS

NAME : AVADUT SANJAY PATIL

USN : P24CO1CA006

SEMESTER/YEAR: II SEM

STUDENT SIGNATURE: _____

GUIDE DETAILS

NAME: _____

DEPARTMENT: _____

DESIGNATION: _____

GUIDE SIGNATURE _____

SUBMISSION DATE	MARKS (Out of 10)

➤ **Abstract :**

This assignment analyzes SQL Injection (SQLi), one of the oldest yet most persistent web vulnerabilities. SQLi occurs when attackers manipulate input fields to inject malicious queries into backend databases, often due to improper input validation or unsafe query construction. Despite decades of awareness, SQLi remains prevalent, ranking high in the OWASP Top 10. The study categorizes attacks into union-based, error-based, blind, time-based, and second-order SQLi, each capable of exposing or altering sensitive data. Real-world incidents—such as the Sony Pictures hack (2011), TalkTalk breach (2015), British Airways payment breach (2018), and the Drupalgeddon vulnerability (2014)—show the widespread damage caused by SQLi, from data theft to large-scale financial fraud. The research highlights defensive measures like parameterized queries, ORM frameworks, Web Application Firewalls (WAFs), and machine learning-based anomaly detection. It concludes that SQLi persists because of insecure coding practices and lack of developer awareness, stressing the need for defense-in-depth, secure SDLC practices, and compliance with global security standards.

➤ Introduction :

SQL Injection (SQLi) is a long-standing web security vulnerability that occurs when an attacker manipulates input fields to inject malicious SQL queries into a backend database. This flaw arises primarily from improper input validation and insecure query construction. Once exploited, SQLi can lead to unauthorized data access, leakage of credentials, deletion of records, or even complete compromise of the database server.

Although widely documented since the early 2000s, SQL Injection remains a critical cyber threat. Modern attackers use automated tools to scan websites for vulnerable parameters, while advanced adversaries craft stealthy queries that bypass weak defenses. SQLi continues to rank high in the OWASP Top 10 vulnerabilities, demonstrating its relevance even in an era of cloud-native and microservice-based applications.

Types of SQL Injection

1. Union-based SQL Injection

- Attackers append malicious UNION SELECT statements to extract information from additional tables.

2. Error-based SQL Injection

- Exploits verbose database error messages to reveal table structures, column names, and system version info.

3. Blind SQL Injection

- Occurs when no error messages are displayed.

4. Time-based Blind SQL Injection

- Relies on deliberate delays (SLEEP() or WAITFOR) to extract data when output is suppressed.

5. Second-Order SQL Injection

- The malicious payload is stored in the database and later executed in a different query context.
- Example: an attacker injects code during registration, which later executes when an admin loads the profile.

➤ **Objectives of Studying SQL Injection :**

- **Understand Core Mechanisms**
 - ✓ Grasp how SQLi arises from unsafe query building and poor sanitization.
- **Identify Attack Vectors**
 - ✓ Recognize vulnerable input points such as login forms, search fields, cookies, and headers.
- **Assess Business Impact**
 - ✓ Evaluate consequences like data theft, financial fraud, reputation loss, and compliance failures.
- **Review Academic and Industrial Research**
 - ✓ Analyze historical and modern countermeasures ranging from query parameterization to machine learning-based anomaly detection.
- **Promote Secure Development**
 - ✓ Encourage developers to use prepared statements, ORM frameworks, and principle of least privilege.
- **Align with Security Standards**
 - ✓ Ensure compliance with OWASP, PCI-DSS, and ISO/IEC standards for application security.
- **Foster Awareness and Training**
 - ✓ Equip organizations and individuals with the knowledge to identify, prevent, and respond to SQLi attempts.
- **Review Academic and Industrial Research**
 - ✓ Analyze historical and modern countermeasures ranging from query parameterization to machine learning-based anomaly detection.

➤ Literature Review :

- ❖ Kindy, D. A., & Pathan, A.-S. K. (2013, originally; updated 2022).
<https://doi.org/10.17762/ijcnis.v5i2.364>
(International Journal of Communication Networks and Information Security, 5(2), detailed survey on SQL Injection vulnerabilities, attacks, and remedies.) [IJCNIS](#)
- ❖ Chaki, S. M. H., & Mat Din, M. (2019, May 31).
<https://doi.org/10.11113/ijic.v9n1.224>
(International Journal of Innovative Computing, 9(1), focused survey on SQL Injection prevention methods.) ijic.utm.my
- ❖ P. S. Saini (2015, June).
<https://doi.org/10.17762/ijritcc.v3i6.4608>
(International Journal on Recent and Innovation Trends in Computing and Communication, 3(6), comparative survey of attacks, detection, and prevention.) ijritcc.org
- ❖ Choudhary, S., Jain, A. K., & Kumar, A. (2017, March).
<https://ijcaonline.org/archives/volume161/number12/27203-2017913411/>
(International Journal of Computer Applications, 161(12), detailed survey on various aspects of SQL Injection.) [IJCA Online](#)
- ❖ Chaki, Siti Hajar; Jaafar, A. G.; & Abdul Rahim, F. (2024, June 28).
<https://doi.org/10.11113/oiji2023.11n2.256>
(Open International Journal of Informatics, 12(1), review of SQL Injection attack and penetration testing process.)

➤ **Real-World Case Studies :**

1. **Yahoo! Breach (2012)** – SQLi exposed 450,000 accounts due to weak validation.
2. **Sony Pictures Hack (2011)** – SQLi allowed attackers to leak user credentials and sensitive company data.
3. **TalkTalk (UK) Breach (2015)** – SQLi exposed personal and financial details of ~157,000 customers.
4. **Heartland Payment Systems (2008)** – SQLi led to malware installation, compromising 130M credit card records.
5. **British Airways Payment Breach (2018)** – Attackers used injection-style flaws to skim 380,000 payment records.
6. **Drupalgeddon (2014)** – SQLi in Drupal CMS allowed attackers to hijack millions of websites.
7. **U.S. Voter Database Leak (2015)** – SQLi exposed 191M voter registration records.
8. **Fortnite Account Breach (2019)** – SQLi flaws in Epic Games servers allowed account takeovers.
9. **Indian Railways Data Breach (2020)** – SQLi led to exposure of passenger records and booking details.
10. **Microsoft SQL Server Exploit Attempts (2021)** – Large-scale SQLi campaigns targeted enterprise apps via botnets.

➤ Conclusion :

SQL Injection remains a persistent cyber threat, even after decades of research, tools, and awareness campaigns. Its longevity stems from the fact that many developers still rely on insecure coding practices such as string concatenation in queries and inadequate input validation. Attackers exploit these weaknesses to bypass authentication, exfiltrate sensitive data, and compromise entire organizations.

Defensive measures have matured, from early research prototypes like SQL rand to industry-standard practices such as parameterized queries, ORM frameworks, and Web Application Firewalls (WAFs). Modern approaches emphasize defense-in-depth, combining automated vulnerability scanning, secure SDLC practices, and penetration testing.

Looking ahead, SQLi continues to adapt in new environments, including NoSQL databases, GraphQL APIs, and cloud-native applications. This broadens the attack surface but also underscores the importance of universal secure coding principles.

Measures have matured, from early research prototypes like SQL rand to industry-standard practices such as parameterized queries, ORM frameworks, and Web Application Firewalls (WAFs). Modern approaches emphasize defense-in-depth, combining automated vulnerability scanning, secure SDLC practices, and penetration testing.

Its longevity stems from the fact that many developers still rely on insecure coding practices such as string concatenation in queries and inadequate input validation. Attackers exploit these weaknesses to bypass authentication, exfiltrate sensitive data, and compromise entire organizations.

SQL Injection is not just a relic of insecure design—it is an ongoing challenge that requires vigilance, secure development training, and continuous monitoring. By adopting layered defenses and aligning with international security standards, organizations can greatly reduce the risk of SQLi and ensure the integrity, confidentiality, and availability of their critical data assets.

➤ **References :**

- ✓ Kindy, D. A., & Pathan, A.-S. K. (2013).
<https://doi.org/10.17762/ijcnis.v5i2.364>
- ✓ Chaki, S. M. H., & Mat Din, M. (2019, May 31).
<https://doi.org/10.11113/ijic.v9n1.224>
- ✓ Saini, P. S. (2015, June).
<https://doi.org/10.17762/ijritcc.v3i6.4608>
- ✓ Choudhary, S., Jain, A. K., & Kumar, A. (2017, March).
<https://ijcaonline.org/archives/volume161/number12/27203-2017913411/>
- ✓ Chaki, S. H., Jaafar, A. G., & Abdul Rahim, F. (2024, June 28).
<https://doi.org/10.11113/oiji2023.11n2.256>